Counting Sort

Counting Sort is a **non-comparison sorting algorithm** used when the input values are integers within a **limited range**. Instead of comparing elements, it counts how many times each value appears.

It is very efficient when the range of numbers is not too large.

1️⃣ What is Counting Sort?

Counting Sort works by:

1. Counting occurrences of each element
2. Computing positions using cumulative counts
3. Placing elements into the correct sorted position

It does **not compare elements**, unlike Bubble, Merge, or Quick sort.

2️⃣ Key Idea

Instead of comparing:

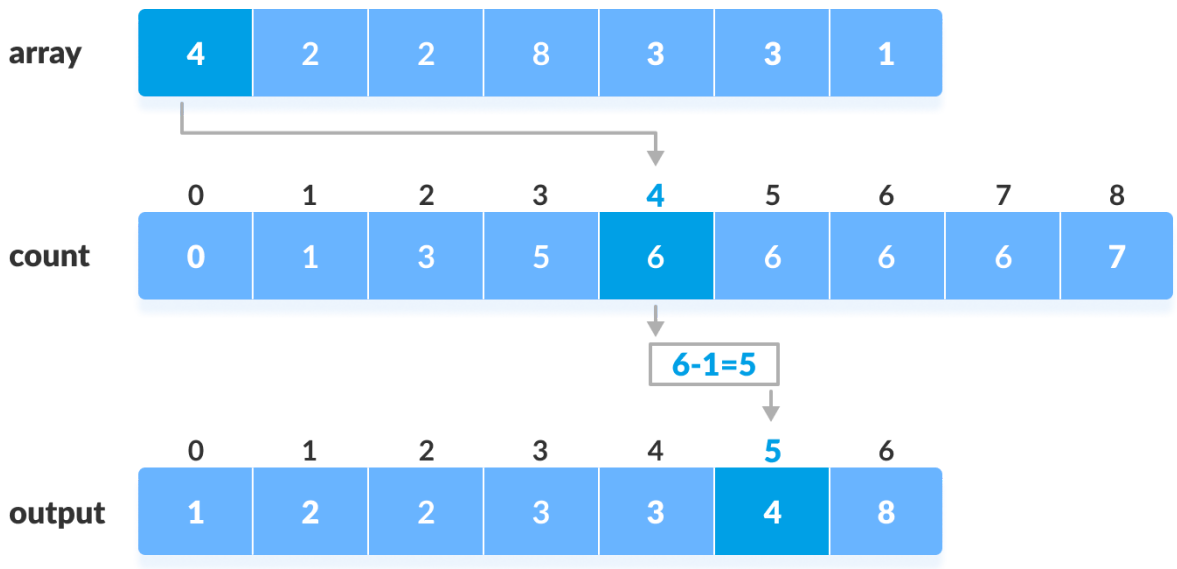
```
8 > 3 ?
```

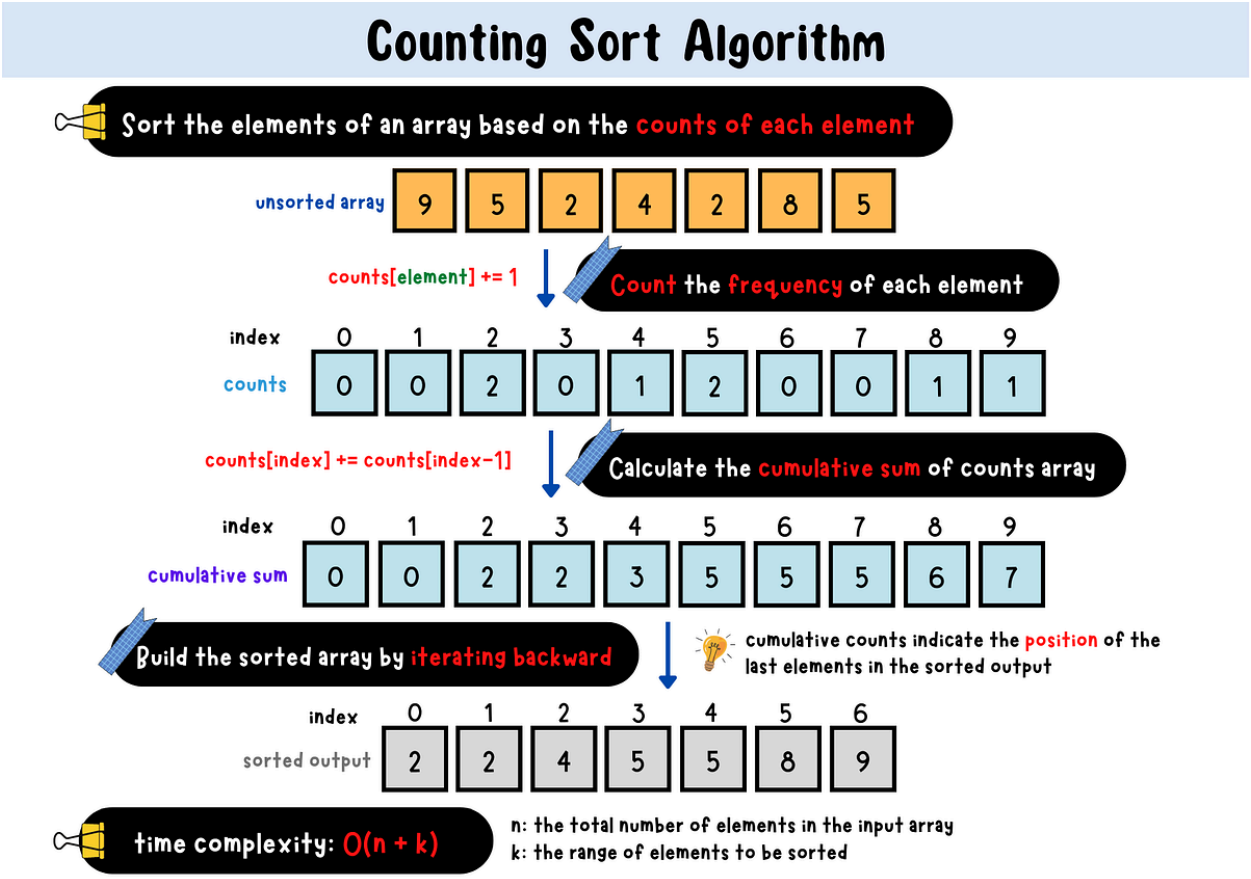
We count:

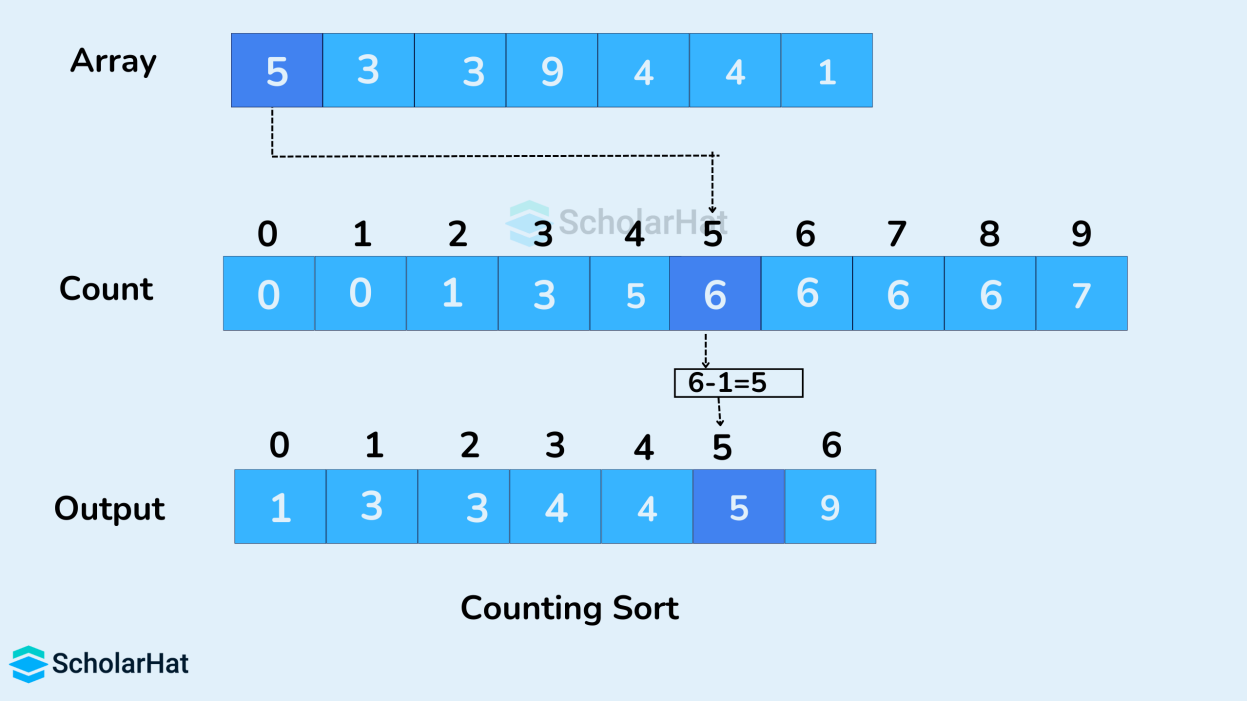
```
How many 1s?
How many 2s?
How many 3s?
```

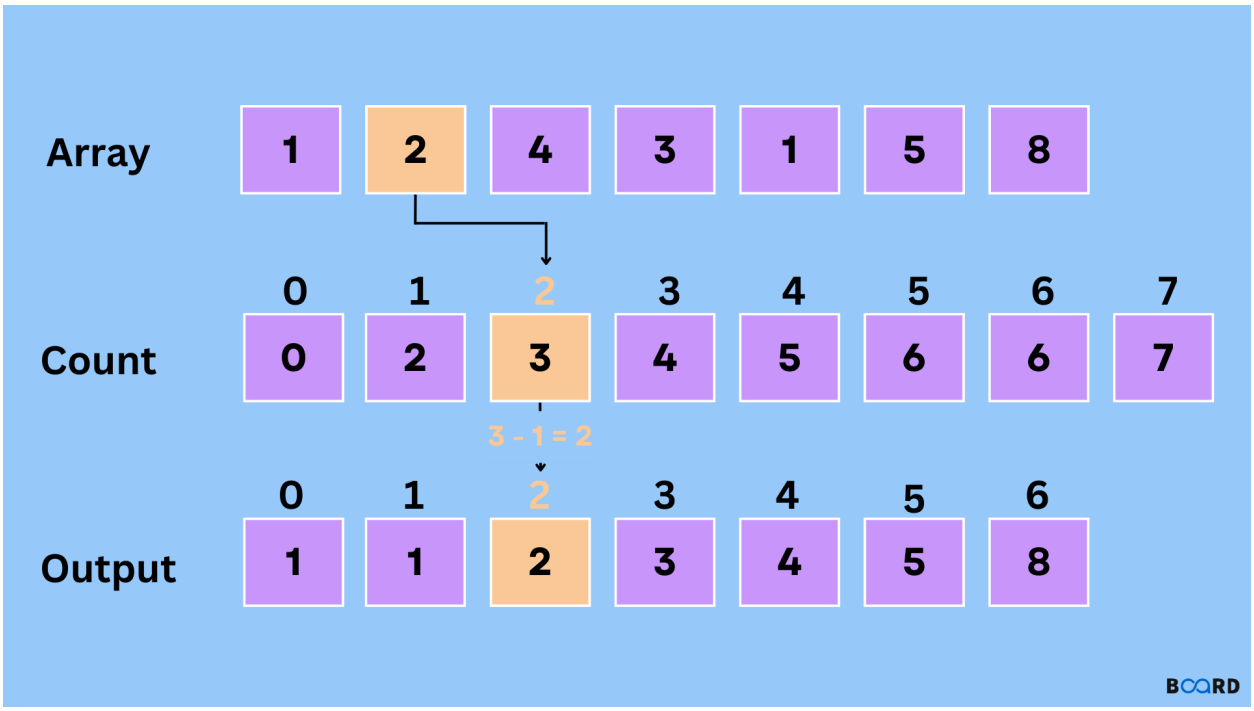
Then reconstruct the sorted array.

3️⃣ Visualization


array
4 2 2 8 3 3 1
0 1 2 3 4 5 6 7 8
count
0 1 3 5 6 6 6 6 7
6-1=5
0 1 2 3 4 5 6
output
1 2 2 3 3 4 8


Counting Sort Algorithm
Sort the elements of an array based on the counts of each element
unsorted array
9 5 2 4 2 8 5
counts[element] += 1
Count the frequency of each element
index
0 1 2 3 4 5 6 7 8 9
counts
0 0 2 0 1 2 0 0 1 1
counts[index] += counts[index-1]
Calculate the cumulative sum of counts array
index
0 1 2 3 4 5 6 7 8 9
cumulative sum
0 0 2 2 3 5 5 5 6 7
Build the sorted array by iterating backward
cumulative counts indicate the position of the last elements in the sorted output
index
0 1 2 3 4 5 6
sorted output
2 2 4 5 5 8 9
time complexity: O(n + k)
n: the total number of elements in the input array
k: the range of elements to be sorted





Example:

```
Input:
4 2 2 8 3 3 1
```

Step 1: Count frequency

```
Value: 1 2 3 4 5 6 7 8
Count: 1 2 2 1 0 0 0 1
```

Step 2: Cumulative count

```
1 3 5 6 6 6 6 7
```

Step 3: Sorted output

```
1 2 2 3 3 4 8
```

4️⃣ When Counting Sort Works Best

Works well when:

```
Range is small
Example: 1 – 100
```

Not good when:

```
Range is huge
Example: 1 – 1,000,000,000
```

5️⃣ Algorithm Steps

Step 1

Find maximum element

Step 2

Create counting array

Step 3

Count frequencies

Step 4

Compute cumulative sum

Step 5

Place elements in output array

6️⃣ Pseudocode

```
COUNTING-SORT(A, n)

find max value k

create count[0..k]

for i = 0 to n-1
    count[A[i]]++

for i = 1 to k
    count[i] += count[i-1]

for i = n-1 down to 0
    output[count[A[i]] - 1] = A[i]
    count[A[i]]--

copy output to A
```

7️⃣ Clean C++ Code

```
#include <iostream>
#include <vector>
using namespace std;

void countingSort(vector<int>& arr) {
    int n = arr.size();

    // Find maximum element
    int maxVal = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }

    // Create counting array
    vector<int> count(maxVal + 1, 0);

    // Count frequencies
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    // Cumulative count
    for (int i = 1; i <= maxVal; i++) {
        count[i] += count[i - 1];
    }

    // Output array
    vector<int> output(n);

    // Place elements in correct position
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
```

```
    // Copy back to original array
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}

void printArray(vector<int>& arr) {
    for (int x : arr)
        cout << x << " ";
    cout << endl;
}

int main() {
    vector<int> arr = {4, 2, 2, 8, 3, 3, 1};

    cout << "Before Sorting: ";
    printArray(arr);

    countingSort(arr);

    cout << "After Sorting: ";
    printArray(arr);

    return 0;
}
```

8️⃣ Step-by-Step Example

Array:

```
4 2 2 3 1
```

Count Array

```
Index: 0 1 2 3 4
Count: 0 1 2 1 1
```

Cumulative

```
0 1 3 4 5
```

Output

```
1 2 2 3 4
```

9️⃣ Time Complexity

Counting Sort runs in:

[

O(n + k)

]

Where:

- n = number of elements
- k = range of input

🔟 Space Complexity

[

O(n + k)

]

Needs extra arrays.

1️⃣1️⃣ Properties

Property	Value
Comparison Sort	❌ No
Stable	✅ Yes
In-place	❌ No
Fast	✅ Yes (small range)

1️⃣2️⃣ Why It Is Stable

Because we insert from **right to left**:

```
for (int i = n-1; i >= 0; i--)
```

This preserves order.

1️⃣3️⃣ Example of Stability

Input:

```
2a 1 2b
```

Output:

```
1 2a 2b
```

Order preserved.

1️⃣4️⃣ Advantages

✅ Very fast

✅ No comparisons

✅ Stable

✅ Simple logic

1️⃣5️⃣ Disadvantages

❌ Extra memory

❌ Only integers

❌ Needs small range

1️⃣6️⃣ Comparison with Other Sorts

Algorithm	Time	Stable	Comparison
Bubble	n^2	Yes	Yes
Merge	n log n	Yes	Yes
Quick	n log n	No	Yes
Counting	n+k	Yes	No

1️⃣7️⃣ When To Use

Good for:

- Exam scores
- Ages
- Small integer ranges

Bad for:

- Large numbers
- Floating points

1️⃣8️⃣ Loop Invariant Idea

After cumulative count:

```
count[i] = position of i
```

After placement:

```
Array sorted
```

1️⃣9️⃣ Common Mistakes

❌ Forgetting cumulative sum

❌ Wrong index placement

❌ Not decreasing count

❌ Iterating forward (breaks stability)

2️⃣0️⃣ Viva Questions

1. Why is counting sort O(n+k)?
2. Why is it stable?
3. Is it comparison sort?
4. When is it inefficient?
5. Space complexity?

2️⃣1️⃣ Short Exam Answer

Counting sort sorts elements by counting occurrences of each value.

Time complexity: O(n+k).

Stable and non-comparison sort.

Works well for small ranges.